

Apply Filters to SQL Queries

Analyst: Mohammad

Tables used: log_in_attempts, employees

Operators applied: AND, OR, NOT, LIKE

Project Description

As a security professional at a large organization, I was tasked with investigating potential security issues involving login attempts and employee machines. Using SQL filters on the organization's log_in_attempts and employees tables, I retrieved specific subsets of data to support the investigation — including after-hours failed logins, logins on suspicious dates, logins originating outside Mexico, and employee machines in specific departments requiring security updates.

Retrieve After Hours Failed Login Attempts

A potential security incident occurred after business hours. I needed to identify all failed login attempts that took place after 18:00.

Query:

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00' AND success = 0;
```

How it works:

- login_time > '18:00' filters for attempts after 6:00 PM
- AND success = 0 additionally requires the attempt to have failed (0 = failed, 1 = succeeded)
- AND means both conditions must be true simultaneously — only after-hours AND failed logins are returned

Retrieve Login Attempts on Specific Dates

A suspicious event occurred on 2022-05-09. I needed to review all login attempts from that day and the day before (2022-05-08).

Query:

```
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

How it works:

- login_date = '2022-05-09' matches attempts on the day of the event
- OR login_date = '2022-05-08' also includes the previous day
- OR means either condition being true is enough — logins from both dates are returned
- Note: the column name must be written in full for each condition even when filtering the same column

Retrieve Login Attempts Outside of Mexico

The team determined suspicious activity did not originate from Mexico. The country column uses both MEX and MEXICO as values.

Query:

```
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

How it works:

- LIKE 'MEX%' matches any country value starting with 'MEX' — covers both MEX and MEXICO
- % is a wildcard that matches any characters following 'MEX'
- NOT negates the condition — only rows where country does NOT start with 'MEX' are returned
- More efficient than writing NOT country = 'MEX' AND NOT country = 'MEXICO' separately

Retrieve Employees in Marketing

Security updates were needed for Marketing department employees in the East building. Office values follow the pattern East- followed by a number.

Query:

```
SELECT *
FROM employees
WHERE department = 'Marketing' AND office LIKE 'East%';
```

How it works:

- department = 'Marketing' filters for Marketing employees only
- AND office LIKE 'East%' additionally requires the office to start with 'East'
- % matches any room number that follows — covers East-170, East-320, etc.
- AND ensures both conditions are met: Marketing employees in East building offices only

Retrieve Employees in Finance or Sales

A different security update was needed for employees in either Finance or Sales departments.

Query:

```
SELECT *
FROM employees
WHERE department = 'Finance' OR department = 'Sales';
```

How it works:

- department = 'Finance' matches Finance employees
- OR department = 'Sales' also matches Sales employees
- OR means either condition is sufficient — both departments returned in one query

Retrieve All Employees Not in IT

All employees outside Information Technology needed a machine update. IT employees had already received it.

Query:

```
SELECT *  
FROM employees  
WHERE NOT department = 'Information Technology';
```

How it works:

- NOT department = 'Information Technology' excludes all IT employees
- Every other department is returned without listing them individually
- More efficient than OR conditions for every other department name

Summary

In this project, I used SQL filters with the AND, OR, NOT, and LIKE operators to investigate two security scenarios: suspicious login activity and employee machines requiring security updates. Across six queries, I filtered the `log_in_attempts` table by time, date, and country, and filtered the `employees` table by department and office location. The LIKE operator with % wildcards was particularly useful for matching partial values such as country codes and building names without requiring exact string matches.